

旁注 Ariane 5——浮点溢出的高昂代价

将大的浮点数转换成整数是一种常见的程序错误来源。1996年6月4日，Ariane 5火箭初次航行，一个错误便产生了灾难性的后果。发射后仅仅37秒钟，火箭偏离了它的飞行路径，解体并且爆炸。火箭上载有价值5亿美元的通信卫星。

后来的调查[73, 33]显示，控制惯性导航系统的计算机向控制引擎喷嘴的计算机发送了一个无效数据。它没有发送飞行控制信息，而是送出了一个诊断位模式，表明在将一个64位浮点数转换成16位有符号整数时，产生了溢出。

溢出的值测量的是火箭的水平速率，这比早先的Ariane 4火箭所能达到的速度高出了5倍。在设计Ariane 4火箭软件时，他们小心地分析了这些数字值，并且确定水平速率决不会超出一个16位数的表示范围。不幸的是，他们在Ariane 5火箭的系统中简单地重用了这一部分，而没有检查它所基于的假设。

 **练习题 2.54** 假定变量 x 、 f 和 d 的类型分别是 `int`、`float` 和 `double`。除了 f 和 d 都不能等于 $+\infty$ 、 $-\infty$ 或者 `NaN`，它们的值是任意的。对于下面每个 C 表达式，证明它总是为真(也就是求值为 1)，或者给出一个使表达式不为真的值(也就是求值为 0)。

- A. `x == (int)(double) x`
- B. `x == (int)(float) x`
- C. `d == (double)(float) d`
- D. `f == (float)(double) f`
- E. `f == -(-f)`
- F. `1.0/2 == 1/2.0`
- G. `d*d >= 0.0`
- H. `(f+d)-f == d`

2.5. 小结

计算机将信息编码为位(比特)，通常组织成字节序列。有不同的编码方式用来表示整数、实数和字符串。不同的计算机模型在编码数字和多字节数据中的字节顺序时使用不同的约定。

C 语言的设计可以包容多种不同字长和数字编码的实现。64 位字长的机器逐渐普及，并正在取代统治市场长达 30 多年的 32 位机器。由于 64 位机器也可以运行 32 位机器编译的程序，我们的重点就放在区分 32 位和 64 位程序，而不是机器本身。64 位程序的优势是可以突破 32 位程序具有的 4GB 地址限制。

大多数机器对整数使用补码编码，而对浮点数使用 IEEE 标准 754 编码。在位级上理解这些编码，并且理解算术运算的数学特性，对于想使编写的程序能在全部数值范围上正确运算的程序员来说，是很重要的。

在相同长度的无符号和有符号整数之间进行强制类型转换时，大多数 C 语言实现遵循的原则是底层的位模式不变。在补码机器上，对于一个 w 位的值，这种行为是由函数 `T2Uw` 和 `U2Tw` 来描述的。C 语言隐式的强制类型转换会出现许多程序员无法预计的结果，常常导致程序错误。

由于编码的长度有限，与传统整数和实数运算相比，计算机运算具有非常不同的属性。当超出表示范围时，有限长度能够引起数值溢出。当浮点数非常接近于 0.0，从而转换成零时，也会下溢。

和大多数其他程序语言一样，C 语言实现的有限整数运算和真实的整数运算相比，有一些特殊的属性。例如，由于溢出，表达式 `x*x` 能够得出负数。但是，无符号数和补码的运算都满足整数运算的许多其他属性，包括结合律、交换律和分配律。这就允许编译器做很多的优化。例如，用 `(x<<3)-x` 取代表达式 `7*x` 时，我们就利用了结合律、交换律和分配律的属性，还利用了移位和乘以 2 的幂之间的关系。

我们已经看到了几种使用位级运算和算术运算组合的聪明方法。例如，使用补码运算，`~x+1` 等价于 `-x`。另外一个例子，假设我们想要一个形如 `[0, ..., 0, 1, ..., 1]` 的位模式，由 $w-k$ 个 0 后面紧跟着 k

个1组成。这些位模式有助于掩码运算。这种模式能够通过C表达式 $(1 \ll k) - 1$ 生成, 利用的是这样一个属性, 即我们想要的位模式的数值为 $2^k - 1$ 。例如, 表达式 $(1 \ll 8) - 1$ 将产生位模式 0xFF。

浮点表示通过将数字编码为 $x \times 2^y$ 的形式来近似地表示实数。最常见的浮点表示方式是由 IEEE 标准 754 定义的。它提供了几种不同的精度, 最常见的是单精度(32 位)和双精度(64 位)。IEEE 浮点也能够表示特殊值 $+\infty$ 、 $-\infty$ 和 NaN。

必须非常小心地使用浮点运算, 因为浮点运算只有有限的范围和精度, 而且并不遵守普遍的算术属性, 比如结合性。

参考文献说明

关于C语言的参考书[45, 61]讨论了不同的数据类型和运算的属性。(这两本书中, 只有 Steele 和 Harbison 的书[45]涵盖了 ISO C99 中的新特性。目前还没有看到任何涉及 ISO C11 新特性的书籍。)对于精确的字长或者数字编码 C 语言标准没有详细的定义。这些细节是故意省去的, 这样可以在更大范围的不同机器上实现 C 语言。已经有几本书[59, 74]给了 C 语言程序员一些建议, 警告他们关于溢出、隐式强制类型转换到无符号数, 以及其他一些已经在这一章中谈及的陷阱。这些书还提供了对变量命名、编码风格和代码测试的有益建议。Seacord 的书[97]是关于 C 和 C++ 程序中的安全问题的, 本书结合了 C 程序的有关信息, 介绍了如何编译和执行程序, 以及漏洞是如何造成的。关于 Java 的书(我们推荐 Java 语言的创始人 James Gosling 参与编写的一本书[5])描述了 Java 支持的数据格式和算术运算。

关于逻辑设计的书[58, 116]都有关于编码和算术运算的章节, 描述了实现算术电路的不同方式。Overton 的关于 IEEE 浮点数的书[82], 从数字应用程序员的角度, 详细描述了格式和属性。

家庭作业

- 2.55 在你能够访问的不同机器上, 使用 show_bytes(文件 show-bytes.c)编译并运行示例代码。确定这些机器使用的字节顺序。
- 2.56 试着用不同的示例值来运行 show_bytes 的代码。
- 2.57 编写程序 show_short、show_long 和 show_double, 它们分别打印类型为 short、long 和 double 的 C 语言对象的字节表示。请试着在几种机器上运行。
- 2.58 编写过程 is_little_endian, 当在小端法机器上编译和运行时返回 1, 在大端法机器上编译运行时则返回 0。这个程序应该可以运行在任何机器上, 无论机器的字长是多少。
- 2.59 编写一个 C 表达式, 它生成一个字, 由 x 的最低有效字节和 y 中剩下的字节组成。对于运算数 $x = 0x89ABCDEF$ 和 $y = 0x76543210$, 就得到 0x765432EF。
- 2.60 假设我们将一个 w 位的字中的字节从 0(最低位)到 $w/8 - 1$ (最高位)编号。写出下面 C 函数的代码, 它会返回一个无符号值, 其中参数 x 的字节 i 被替换成字节 b:

```
unsigned replace_byte (unsigned x, int i, unsigned char b);
```

以下示例, 说明了这个函数该如何工作:

```
replace_byte(0x12345678, 2, 0xAB) --> 0x12AB5678
replace_byte(0x12345678, 0, 0xAB) --> 0x123456AB
```

位级整数编码规则

在接下来的作业中, 我们特意限制了你能使用的编程结构, 来帮你更好地理解 C 语言的位级、逻辑和算术运算。在回答这些问题时, 你的代码必须遵守以下规则:

- 假设
 - 整数用补码形式表示。
 - 有符号数的右移是算术右移。
 - 数据类型 int 是 w 位长的。对于某些题目, 会给定 w 的值, 但是在其他情况下, 只要 w 是 8 的整数倍, 你的代码就应该能工作。你可以用表达式 $\text{sizeof(int)} \ll 3$ 来计算 w。